EXPERIMENT 01

Regular Expression Matching and Searching

Aim

To demonstrate how regular expressions can match and search for patterns in text using Python.

Procedure

1. Import the re module.
2. Create a sample text and a regular expression pattern.
3. Use re.search() to find the first match.
4. Use re.findall() to collect all matches.
5. Display the results.

Program

```
import re

text = "Contact us at help@example.com or sales@example.org"
pattern = r"[\w.-]+@[\w.-]+\.\w+"

first_match = re.search(pattern, text)
all_matches = re.findall(pattern, text)

print("First email:", first_match.group() if first_match else "Not found")
print("All emails:", all_matches)
```

Output

```
OUTPUT — EXPERIMENT 01
First email: help@example.com
All emails: ['help@example.com', 'sales@example.org']
```

Result

Thus, regular expressions were used successfully to search and match patterns in text.

EXPERIMENT 02

Finite State Automaton for Strings Ending with 'ab'

Aim

To implement a basic finite-state automaton that accepts strings ending with the pattern 'ab'.

Procedure

1. Define states representing the progress toward the ending pattern.
2. Read each character and update the state.
3. Accept the string if the final state represents a suffix 'ab'.
4. Test the automaton with sample strings.

Program

```
def ends_with_ab(text):
    state = 0
    for ch in text:
        if state == 0:
            state = 1 if ch == "a" else 0
        elif state == 1:
            state = 2 if ch == "b" else (1 if ch == "a" else 0)
        else:  # current suffix is "ab"
            state = 1 if ch == "a" else 0
    return state == 2

for value in ["ab", "aab", "abab", "aba", "abb"]:
    print(value, "->", "Accepted" if ends_with_ab(value) else "Rejected")
```

Output

```
OUTPUT — EXPERIMENT 02
ab -> Accepted
aab -> Accepted
abab -> Accepted
aba -> Rejected
abb -> Rejected
```

Result

Thus, the finite-state automaton accepted strings ending with 'ab' and rejected the others.

EXPERIMENT 03

Morphological Analysis Using NLTK

Aim

To demonstrate basic morphological analysis by tokenizing words and identifying their stems using NLTK.

Procedure

1. Install NLTK using pip if required.
2. Import NLTK's PorterStemmer.
3. Prepare a list of words.
4. Apply stemming to each word.
5. Display each word with its stem.

Program

```
# Install first: pip install nltk
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["playing", "played", "plays", "studies", "happily"]

for word in words:
    print(word, "->", stemmer.stem(word))
```

Output

```
OUTPUT — EXPERIMENT 03
playing -> play
played -> play
plays -> play
studies -> studi
happily -> happili
```

Result

Thus, NLTK was used to demonstrate a basic form of morphological analysis through stemming.

EXPERIMENT 04

Finite-State Machine for Plural Noun Forms

Aim

To implement a simple finite-state-style rule system that generates plural forms of English nouns.

Procedure

1. Define rules for nouns ending in s, x, ch, or sh.
2. Add 'es' for these endings.
3. Change consonant plus y to 'ies'.
4. Add 's' for other nouns.
5. Test the rules using sample nouns.

Program

```
def plural(noun):
    lower = noun.lower()
    if lower.endswith(("s", "x", "ch", "sh")):
        return noun + "es"
    if lower.endswith("y") and len(noun) > 1 and lower[-2] not in "aeiou":
        return noun[:-1] + "ies"
    return noun + "s"

for noun in ["cat", "box", "church", "baby", "book"]:
    print(noun, "->", plural(noun))
```

Output

```
OUTPUT — EXPERIMENT 04
cat -> cats
box -> boxes
church -> churches
baby -> babies
book -> books
```

Result

Thus, a rule-based finite-state-style machine generated plural forms for the sample nouns.

EXPERIMENT 05

Word Stemming Using the Porter Stemmer

Aim

To use the Porter Stemmer algorithm to reduce words to their stems.

Procedure

1. Import PorterStemmer from NLTK.
2. Create a stemmer object.
3. Prepare a list of words with different endings.
4. Stem each word.
5. Print the original word and stem.

Program

```
# Install first: pip install nltk
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ["connect", "connected", "connecting", "connection", "relational"]

for word in words:
    print(word, "->", stemmer.stem(word))
```

Output

```
OUTPUT — EXPERIMENT 05
connect -> connect
connected -> connect
connecting -> connect
connection -> connect
relational -> relat
```

Result

Thus, the Porter Stemmer was applied successfully to reduce words to their stems.

EXPERIMENT 06

Bigram N-Gram Text Generation

Aim

To implement a basic bigram model that predicts the next word from the current word and generates text.

Procedure

1. Prepare a small training sentence.
2. Split the sentence into words.
3. Build a dictionary of word-to-next-word transitions.
4. Start from a chosen word and follow the transitions.
5. Print the generated sequence.

Program

```
import random

text = "we learn python and we learn language models"
words = text.split()
bigrams = {}
for first, second in zip(words, words[1:]):
    bigrams.setdefault(first, []).append(second)

current = "we"
generated = [current]
for _ in range(8):
    choices = bigrams.get(current)
    if not choices:
        break
    current = random.choice(choices)
    generated.append(current)

print("Generated text:", " ".join(generated))
```

Output

```
OUTPUT — EXPERIMENT 06
Generated text: we learn python and we learn language models
(Note: output may vary because the next word is selected randomly.)
```

Result

Thus, a simple bigram model generated text by selecting words based on learned word transitions.

EXPERIMENT 07

Part-of-Speech Tagging Using NLTK

Aim

To tag words with their parts of speech using the NLTK library.

Procedure

1. Install NLTK if required.
2. Import word tokenization and POS-tagging functions.
3. Tokenize the sample sentence.
4. Run the POS tagger.
5. Display the tagged words.

Program

```
# Install first: pip install nltk
import nltk
from nltk import word_tokenize, pos_tag

# Run once if resources are missing:
# nltk.download("punkt")
# nltk.download("punkt_tab")
# nltk.download("averaged_perceptron_tagger_eng")

sentence = "The small cat sleeps peacefully."
tokens = word_tokenize(sentence)
print(pos_tag(tokens))
```

Output

```
OUTPUT — EXPERIMENT 07
[('The', 'DT'), ('small', 'JJ'), ('cat', 'NN'),
 ('sleeps', 'VBZ'), ('peacefully', 'RB'), ('.', '.')]
(Exact tags can vary with NLTK model versions.)
```

Result

Thus, NLTK assigned part-of-speech tags to the words in the sentence.

EXPERIMENT 08
Stochastic Part-of-Speech Tagging

Aim

To demonstrate probabilistic POS tagging by choosing the most likely tag for each word from a small model.

Procedure

1. Create a small dictionary of word-tag probabilities.
2. Choose the tag with the highest probability for each word.
3. Use a default tag for unknown words.
4. Display each word with its selected tag.

Program

```
tag_probabilities = {
    "time": {"NN": 0.8, "VB": 0.2},
    "flies": {"NNS": 0.4, "VBZ": 0.6},
    "like": {"IN": 0.7, "VB": 0.3},
    "birds": {"NNS": 0.9, "VBZ": 0.1}
}

sentence = "time flies like birds".split()
for word in sentence:
    probabilities = tag_probabilities.get(word.lower(), {"NN": 1.0})
    tag = max(probabilities, key=probabilities.get)
    print(word, "->", tag)
```

Output

```
OUTPUT — EXPERIMENT 08
time -> NN
flies -> VBZ
like -> IN
birds -> NNS
```

Result

Thus, a simple stochastic model assigned POS tags using the highest probability.

EXPERIMENT 09

Rule-Based POS Tagging Using Regular Expressions

Aim

To implement a rule-based part-of-speech tagger using regular-expression patterns.

Procedure

1. Import RegexpTagger from NLTK.
2. Define suffix-based tagging patterns.
3. Tokenize a sample sentence.
4. Apply the regular-expression tagger.
5. Print the tagged tokens.

Program

```
# Install first: pip install nltk
from nltk import RegexpTagger, word_tokenize

patterns = [
    (r".*ing$", "VBG"),
    (r".*ed$", "VBD"),
    (r".*ly$", "RB"),
    (r".*s$", "NNS"),
    (r".*", "NN")
]
tagger = RegexpTagger(patterns)
sentence = "Birds are flying quickly".split()
print(tagger.tag(sentence))
```

Output

```
OUTPUT — EXPERIMENT 09
[('Birds', 'NNS'), ('are', 'NNS'), ('flying', 'VBG'), ('quickly', 'RB')]
```

Result

Thus, regular-expression rules were used to assign approximate POS tags to words.

EXPERIMENT 10
Transformation-Based Tagging

Aim

To demonstrate transformation-based tagging by correcting an initial tag using a simple rule.

Procedure

1. Assign initial tags to the words.
2. Define a transformation rule.
3. Check each word against the rule context.
4. Replace the tag when the rule applies.
5. Display the corrected tags.

Program

```
tokens = [("the", "NN"), ("dogs", "NNS"), ("run", "NN")]

# Transformation rule: "the" is tagged as a determiner.
for i, (word, tag) in enumerate(tokens):
    if word.lower() == "the":
        tokens[i] = (word, "DT")

print(tokens)
```

Output

```
OUTPUT — EXPERIMENT 10
[('the', 'DT'), ('dogs', 'NNS'), ('run', 'NN')]
```

Result

Thus, a transformation rule corrected an initial tag in the sample sequence.

EXPERIMENT 11

Top-Down Parser for a Context-Free Grammar

Aim

To implement a simple top-down parser that checks whether a sentence can be generated by a context-free grammar.

Procedure

1. Define grammar rules for a simple sentence.
2. Create a parser that expands grammar symbols from the start symbol.
3. Match terminal symbols against input words.
4. Accept when all words and grammar symbols are consumed.
5. Test the sample sentence.

Program

```
# A compact recursive top-down parser for: S -> NP VP
grammar = {
    "S": [["NP", "VP"]],
    "NP": [["Det", "N"]],
    "VP": [["V"]],
    "Det": [["the"]],
    "N": [["cat"], ["dog"]],
    "V": [["sleeps"], ["runs"]]
}

def parse(symbol, words, pos):
    if symbol not in grammar:
        return pos + 1 if pos < len(words) and words[pos] == symbol else None
    for rule in grammar[symbol]:
        next_pos = pos
        for part in rule:
            result = parse(part, words, next_pos)
            if result is None:
                break
            next_pos = result
        else:
            return next_pos
    return None

sentence = "the cat sleeps".split()
print("Accepted" if parse("S", sentence, 0) == len(sentence) else "Rejected")
```

Output

```
OUTPUT — EXPERIMENT 11
Accepted
```

Result

Thus, the top-down parser checked the sentence against the defined context-free grammar.

EXPERIMENT 12

Earley Parser for a Context-Free Grammar

Aim

To use an Earley parser to determine whether a sentence is accepted by a context-free grammar.

Procedure

1. Install NLTK if required.
2. Define a context-free grammar.
3. Create an Earley chart parser.
4. Parse the sample sentence.
5. Display whether a parse is found.

Program

```
# Install first: pip install nltk
from nltk import CFG
from nltk.parse import EarleyChartParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V
Det -> 'the'
N -> 'cat' | 'dog'
V -> 'sleeps' | 'runs'
""")
parser = EarleyChartParser(grammar)
sentence = "the cat sleeps".split()
trees = list(parser.parse(sentence))
print("Parse found" if trees else "No parse found")
if trees:
    print(trees[0])
```

Output

```
OUTPUT — EXPERIMENT 12
Parse found
(S (NP (Det the) (N cat)) (VP (V sleeps)))
```

Result

Thus, the Earley parser successfully found a parse for the sentence.

EXPERIMENT 13

Parse Tree Generation Using Context-Free Grammar

Aim

To generate and display a parse tree for a sentence using a context-free grammar.

Procedure

1. Import CFG and RecursiveDescentParser from NLTK.
2. Define the grammar rules.
3. Provide the input sentence as tokens.
4. Generate the parse tree.
5. Print the tree representation.

Program

```
# Install first: pip install nltk
from nltk import CFG
from nltk.parse import RecursiveDescentParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V
Det -> 'the'
N -> 'student'
V -> 'reads'
""")
parser = RecursiveDescentParser(grammar)
for tree in parser.parse("the student reads".split()):
    print(tree)
    tree.pretty_print()
```

Output

```
OUTPUT — EXPERIMENT 13
(S (NP (Det the) (N student)) (VP (V reads)))
A tree diagram is printed in the terminal.
```

Result

Thus, a parse tree was generated for the sentence using the given grammar.

EXPERIMENT 14

Sentence Agreement Checking

Aim

To check simple subject-verb agreement using rules based on a small context-free grammar.

Procedure

1. Define allowed subject and verb forms.
2. Split the sentence into words.
3. Check whether the subject and verb agree in number.
4. Display the agreement result.
5. Test singular and plural examples.

Program

```
def check_agreement(sentence):
    words = sentence.lower().replace(".", "").split()
    if len(words) != 2:
        return "Unsupported sentence pattern"
    subject, verb = words
    singular_subjects = {"he", "she", "it", "cat"}
    plural_subjects = {"they", "we", "dogs"}
    singular_verbs = {"runs", "sleeps"}
    plural_verbs = {"run", "sleep"}

    if subject in singular_subjects and verb in singular_verbs:
        return "Agreement correct"
    if subject in plural_subjects and verb in plural_verbs:
        return "Agreement correct"
    return "Agreement incorrect"

for s in ["he runs", "they run", "he run"]:
    print(s, ":", check_agreement(s))
```

Output

```
OUTPUT — EXPERIMENT 14
he runs : Agreement correct
they run : Agreement correct
he run : Agreement incorrect
```

Result

Thus, the program checked subject-verb agreement for the defined sentence patterns.

EXPERIMENT 15

Probabilistic Context-Free Grammar Parsing

Aim

To parse a sentence using a probabilistic context-free grammar and select a likely parse.

Procedure

1. Install NLTK if required.
2. Define a probabilistic context-free grammar with probabilities.
3. Create a Viterbi parser.
4. Parse the input sentence.
5. Display the parse tree and probability.

Program

```
# Install first: pip install nltk
from nltk import PCFG
from nltk.parse import ViterbiParser

grammar = PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.7] | N [0.3]
VP -> V [1.0]
Det -> 'the' [1.0]
N -> 'student' [0.6] | 'teacher' [0.4]
V -> 'studies' [1.0]
""")
parser = ViterbiParser(grammar)
for tree in parser.parse("the student studies".split()):
    print(tree)
    print("Probability:", tree.prob())
```

Output

```
OUTPUT — EXPERIMENT 15
(S (NP (Det the) (N student)) (VP (V studies)))
Probability: 0.42
```

Result

Thus, the Viterbi parser found a likely parse using the probabilities in the PCFG.

EXPERIMENT 16

Named Entity Recognition Using spaCy

Aim

To identify named entities such as people, organizations, and locations using spaCy.

Procedure

1. Install spaCy and its English model.
2. Load the English language model.
3. Provide a sample text.
4. Process the text using spaCy.
5. Print each entity and its label.

Program

```
# Install: pip install spacy
# Then run: python -m spacy download en_core_web_sm
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Sundar Pichai works at Google in California.")

for entity in doc.ents:
    print(entity.text, "->", entity.label_)
```

Output

```
OUTPUT — EXPERIMENT 16
Sundar Pichai -> PERSON
Google -> ORG
California -> GPE
(Labels can vary slightly by model version.)
```

Result

Thus, spaCy was used to identify named entities in the input text.

EXPERIMENT 17

Accessing WordNet and Exploring Word Meanings

Aim

To access WordNet and retrieve synsets and definitions for a given word.

Procedure

1. Install NLTK if required.
2. Download the WordNet corpus once.
3. Import the WordNet interface.
4. Retrieve synsets for a sample word.
5. Print each synset and its definition.

Program

```
# Install first: pip install nltk
import nltk
from nltk.corpus import wordnet as wn

nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

word = "bank"
for synset in wn.synsets(word)[:5]:
    print(synset.name(), ":", synset.definition())
```

Output

```
OUTPUT — EXPERIMENT 17
bank.n.01 : sloping land (especially the slope beside a body of water)
bank.n.02 : a financial institution that accepts deposits
(Definitions/order are supplied by the installed WordNet version.)
```

Result

Thus, WordNet synsets and definitions were retrieved to explore different word meanings.

EXPERIMENT 18
First-Order Predicate Logic (FOPC) Parser

Aim

To implement a simple parser for basic first-order predicate logic expressions.

Procedure

1. Define a pattern for predicate expressions.
2. Recognize an optional negation symbol.
3. Extract the predicate name and its arguments.
4. Validate the expression using a regular expression.
5. Display the parsing result.

Program

```
import re

pattern = r"\s*(~?)([A-Za-z][A-Za-z0-9_]*)\(([^()]*)\)\s*"

def parse_fopc(expression):
    match = re.fullmatch(pattern, expression)
    if not match:
        return "Invalid expression"
    negation, predicate, args = match.groups()
    arguments = [item.strip() for item in args.split(",") if item.strip()]
    return {
        "negated": bool(negation),
        "predicate": predicate,
        "arguments": arguments
    }

print(parse_fopc("Likes(Alice, Bob)"))
print(parse_fopc("~Human(Socrates)"))
```

Output

```
OUTPUT — EXPERIMENT 18
{'negated': False, 'predicate': 'Likes', 'arguments': ['Alice', 'Bob']}
{'negated': True, 'predicate': 'Human', 'arguments': ['Socrates']}
```

Result

Thus, a simple FOPC parser extracted predicate names, arguments, and negation.

EXPERIMENT 19

Word Sense Disambiguation Using the Lesk Algorithm

Aim

To identify the likely meaning of an ambiguous word using the Lesk algorithm.

Procedure

1. Install NLTK and download WordNet.
2. Import the Lesk function.
3. Prepare a sentence containing an ambiguous word.
4. Apply Lesk to the sentence and target word.
5. Print the selected synset and definition.

Program

```
# Install first: pip install nltk
import nltk
from nltk.wsd import lesk
from nltk.corpus import wordnet as wn

nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

sentence = "He deposited money in the bank".split()
sense = lesk(sentence, "bank")
print("Selected sense:", sense)
if sense:
    print("Meaning:", sense.definition())
```

Output

```
OUTPUT — EXPERIMENT 19
Selected sense: a WordNet synset for 'bank'
Meaning: definition of the selected synset
(The exact sense depends on the installed WordNet data and Lesk
    implementation.)
```

Result

Thus, the Lesk algorithm was used to select a likely sense of the ambiguous word from context.

EXPERIMENT 20
Information Retrieval Using TF-IDF

Aim

To rank documents according to their relevance to a query using TF-IDF.

Procedure

1. Prepare a small collection of documents.
2. Create a TF-IDF vectorizer.
3. Transform the documents into vectors.
4. Transform the search query into a vector.
5. Calculate cosine similarity and rank the documents.

Program

```
# Install first: pip install scikit-learn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Python is useful for data science",
    "Football is a popular sport",
    "Python supports machine learning"
]
query = ["Python machine learning"]

vectorizer = TfidfVectorizer()
matrix = vectorizer.fit_transform(documents + query)
scores = cosine_similarity(matrix[-1], matrix[:-1]).flatten()

for index in scores.argsort()[::-1]:
    print(f"Document {index + 1}: score={scores[index]:.3f}")
```

Output

```
OUTPUT — EXPERIMENT 20
Document 3: score=0.5...
Document 1: score=0.2...
Document 2: score=0.0
(Exact scores depend on the TF-IDF implementation.)
```

Result

Thus, documents were ranked by their TF-IDF vector similarity to the query.

EXPERIMENT 21

Syntax-Driven Semantic Analysis

Aim

To perform simple syntax-driven semantic analysis by extracting noun phrases and assigning basic meanings.

Procedure

1. Define a small grammar for noun phrases and sentences.
2. Parse the sentence using NLTK.
3. Extract noun phrase subtrees.
4. Assign a simple semantic label to each noun phrase.
5. Display the extracted phrases and meanings.

Program

```
# Install first: pip install nltk
from nltk import CFG
from nltk.parse import ChartParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'student' | 'book' | 'teacher'
V -> 'reads' | 'helps'
""")
parser = ChartParser(grammar)
sentence = "the student reads a book".split()

for tree in parser.parse(sentence):
    for subtree in tree.subtrees(lambda t: t.label() == "NP"):
        phrase = " ".join(subtree.leaves())
        meaning = "person/entity" if "student" in phrase or "teacher" in phrase else
"object"
        print(phrase, "->", meaning)
    break
```

Output

```
OUTPUT — EXPERIMENT 21
the student -> person/entity
a book -> object
```

Result

Thus, noun phrases were extracted from the parse tree and assigned simple semantic labels.

EXPERIMENT 22
Reference Resolution in Text

Aim

To implement a simple reference-resolution program that links pronouns to likely preceding nouns.

Procedure

1. Tokenize a short text into sentences and words.
2. Collect candidate nouns from the text.
3. For each pronoun, find the nearest preceding candidate noun.
4. Display the pronoun and its proposed referent.
5. Note that this is a simplified heuristic.

Program

```
import re

text = "Ravi bought a laptop. He uses it every day."
sentences = re.split(r"(?<=[.!?])\s+", text)
candidates = []

for sentence in sentences:
    words = re.findall(r"\b[A-Za-z]+\b", sentence)
    for word in words:
        if word.lower() not in {"he", "she", "it", "they", "a", "the", "uses", "bought",
"every", "day"}:
            candidates.append(word)
    for pronoun in [w for w in words if w.lower() in {"he", "she", "it", "they"}]:
        referent = candidates[-1] if candidates else "unknown"
        print(pronoun, "->", referent)
```

Output

```
OUTPUT — EXPERIMENT 22
He -> laptop
it -> laptop
(Note: this simple heuristic can be wrong; it is not a full coreference
    system.)
```

Result

Thus, a basic heuristic was used to propose referents for pronouns in the text.

EXPERIMENT 23

Text Coherence Evaluation

Aim

To evaluate simple text coherence using sentence-to-sentence word overlap.

Procedure

1. Prepare a short multi-sentence text.
2. Split the text into sentences.
3. Convert each sentence to a set of words.
4. Calculate overlap between adjacent sentence word sets.
5. Display average overlap as a simple coherence indicator.

Program

```
import re

text = ("Python is a programming language. "
        "Python supports many programming tasks. "
        "Many tasks require careful planning.")
sentences = [set(re.findall(r"\b\w+\b", s.lower()))
             for s in re.split(r"(?<=[.!?])\s+", text)]
scores = []
for first, second in zip(sentences, sentences[1:]):
    union = first | second
    scores.append(len(first & second) / len(union) if union else 0)

average = sum(scores) / len(scores) if scores else 0
print("Adjacent sentence overlaps:", [round(s, 3) for s in scores])
print("Simple coherence score:", round(average, 3))
```

Output

```
OUTPUT — EXPERIMENT 23
Adjacent sentence overlaps: [a numeric score, a numeric score]
Simple coherence score: a numeric score
(The program calculates the exact scores when run.)
```

Result

Thus, adjacent-sentence word overlap was calculated as a basic indicator of text coherence.

EXPERIMENT 24
Dialog Act Recognition

Aim

To recognize basic dialog acts such as greeting, question, thanks, and farewell using rules.

Procedure

1. Define keywords and patterns for common dialog acts.
2. Read each utterance in a conversation.
3. Match the utterance against the rules.
4. Assign a dialog-act label.
5. Display each utterance with its label.

Program

```
def dialog_act(utterance):
    text = utterance.lower().strip()
    if any(word in text for word in ["hello", "hi", "good morning"]):
        return "GREETING"
    if text.endswith("?") or text.startswith(("who", "what", "where", "why", "how")):
        return "QUESTION"
    if "thank" in text:
        return "THANKS"
    if any(word in text for word in ["bye", "goodbye", "see you"]):
        return "FAREWELL"
    return "STATEMENT"

dialog = ["Hello there!", "Where is the library?", "Thank you", "Goodbye"]
for utterance in dialog:
    print(utterance, "->", dialog_act(utterance))
```

Output

```
OUTPUT — EXPERIMENT 24
Hello there! -> GREETING
Where is the library? -> QUESTION
Thank you -> THANKS
Goodbye -> FAREWELL
```

Result

Thus, rule-based dialog-act labels were assigned to the sample conversation.

EXPERIMENT 25

Text Generation Using GPT-3 / OpenAI API

Aim

To generate text from a prompt using an OpenAI language model API.

Procedure

1. Install the OpenAI Python package.
2. Create an API key in the OpenAI platform and store it as an environment variable.
3. Use the SDK to send a prompt to an available text-generation model.
4. Print the generated response.
5. Keep API keys private and check model access and billing.

Program

```
# Install: pip install openai
# Set OPENAI_API_KEY in your environment before running.
from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model="gpt-3.5-turbo-instruct",
    input="Write three sentences about protecting the environment."
)
print(response.output_text)
```

Output

```
OUTPUT — EXPERIMENT 25

Example output:
Protecting the environment keeps our air and water clean.
Reducing waste conserves valuable resources.
Planting trees supports wildlife and helps absorb carbon dioxide.
(Note: output is generated by the API and may vary.)
```

Result

Thus, a prompt was sent to an OpenAI text-generation API and the returned text was displayed. Model availability may depend on the account and API.

EXPERIMENT 26

English-to-French Machine Translation Using Transformers

Aim

To translate English text into French using a pretrained Hugging Face Transformers model.

Procedure

1. Install transformers and sentencepiece.
2. Load a pretrained English-to-French translation model.
3. Tokenize the English input sentence.
4. Generate the French translation.
5. Decode and display the translated text.

Program

```
# Install: pip install transformers sentencepiece torch
from transformers import pipeline

translator = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-en-fr"
)
text = "Hello, how are you?"
result = translator(text, max_length=100)
print("English:", text)
print("French:", result[0]["translation_text"])
```

Output

```
OUTPUT — EXPERIMENT 26
English: Hello, how are you?
French: Bonjour, comment allez-vous ?
(Note: exact translation may vary by model version.)
```

Result

Thus, the pretrained Transformers model translated the English sentence into French.